CMPSC 448 Midterm Project Report

Nick Foulkrod

Section 1: Problem Definition

With Artificial Intelligence advancing so rapidly within the past few years, it has become increasingly important to be able to tell not only when text has been AI-generated, but also which model created it. This project focuses on a less commonly researched problem, not just determining if machine learning models can detect AI generated text, but determining if they can identify which LLM created it. I have employed two models, a Convolutional Neural Network (CNN), and a bi-directional Recurrent Neural Network using Long Short-Term Memory (LSTM), on three families of LLM, OpenAI, Llama, and Mistral. Both models need to be able to identify each family with greater than 33% accuracy (that of random guessing).

To test this, I have considered four research questions:

1. Can we identify which LLM generated a response?

2. Does the user's prompt help identify the LLM?

3. Do LLM fingerprints generalize across tasks or domains?

4. What characteristics distinguish different LLMs?

Section 2: Data Curation

The dataset I decided to use is the RAID dataset, created by Liam Dugan and colleagues at the University of Pennsylvania, designed to test the efficacy of AI text-detection services. This dataset is incredibly useful for this project because it focuses on an extremely similar problem. As a result, very little modification was required on my part. I downloaded this dataset from the Hugging Face Datasets Hub, which serves as a massive repository of data for machine learning problems such as this.

RAID contains several million rows of training data, consisting of prompts directing each model to generate the text within 8 different media domains: abstracts, books, news, poetry, recipes, Reddit posts, reviews, and Wikipedia articles. These outputs have been generated by 12 separate models, listed in Table 1. For the purposes of this project, I decided to focus on the three most well-known model families. Notably, I excluded gpt2 from OpenAI-GPT, as it was by far the oldest model within the data.

RAID's data is intended to be used to test AI text detection-services' ability to withstand specific modifications to generated outputs, such as rewording, surface edits, and formatting changes. To keep my analysis focused entirely on unaltered LLM outputs, I only included the rows with attack == "none".

To collect my data, I scanned the first 3 million rows of the dataset. For each of the four domains, I collected 400 rows per family and divided that amount equally among the models within the family: 400 rows from llama-chat, 200 each from mistral and mistral-chat, and 133 each from chatgpt, gpt4, and gpt3. Because 400 does not divide evenly by three, the OpenAI-GPT family has 399 rows per domain, for a total of 4,796 rows. Without this even split, all rows would likely come from the same model, which would change the problem from family identification to model identification.

Family	RAID model values	Status	Abstracts	Books	News	Poetry	Total
OpenAI-GPT	chatgpt, gpt4, gpt3 (133 each)	Included	399	399	399	399	1,596
Meta-Llama	llama-chat (400)	Included	400	400	400	400	1,600
Mistral	mistral, mistral-chat (200 each)	Included	400	400	400	400	1,600
Cohere	cohere, cohere-chat	Excluded	–	–	–	–	–
MosaicML-MPT	mpt, mpt-chat	Excluded	–	–	–	–	–
GPT-2	gpt2	Excluded	–	–	–	–	–
Human	human	Excluded	–	–	–	–	–
Total			1,199	1,199	1,199	1,199	4,796

Table 1: Model Families and Rows per Domain (rows per model in parentheses)

4 domains (Recipes, Reddit posts, Reviews, and Wikipedia Articles) were excluded due to not being included within the first 3 million rows. 4 separate domains should be sufficient to test between domains.

Within the data, I assigned 70% by source document to training, 15% to validation, and 15% to testing.

Section 3: Model/Training Details

Setting	CNN	LSTM
Model	TextCNN	BiLSTM
Tokenizer	Word-level, punctuation and line breaks kept as tokens, case preserved	same
Vocabulary	20,000 most frequent training tokens	same
Max sequence length	64 tokens for input, 256 for output	same
Embedding	128 dimensions, trained from scratch	same
Architecture	3 parallel 1-D convolutions, kernel widths 3, 4, 5, with 100 filters each	1-layer bidirectional LSTM, hidden size 128 per direction
Pooling	ReLU, then max over time	Final hidden states of both directions concatenated
Dropout	0.5 before the output layer	same
Output layer	Linear, 300 to number of families	Linear, 256 to number of families
Optimizer	Adam, learning rate 0.001	same
Loss	Cross-entropy	same
Batch size	64	same
Epochs	12, keeping the epoch with the best validation accuracy	same
Gradient clipping	Max norm 5.0	Same
Random seed	42	Same

Table 2: Hyperparameters

I decided to keep punctuation and line breaks from the original outputs, as those are important style signals that could differ between models. Instead of including the full original outputs from the dataset, I cut off the outputs at 256 tokens and the inputs at 64 tokens to allow for faster training. My CNN's kernel widths of 3, 4, and 5 were chosen such that each filter represents a phrase and max pooling can determine whether that phrase existed anywhere within the output.

LSTM's bidirectional nature means that it can read the whole output in order from front to back and vice versa, with greater emphasis placed on the beginning and end of each output. Finally, I decided to set the number of epochs to 12, as CNN tended to take many epochs to learn. I kept the best epoch for each model to avoid overtraining any one model.

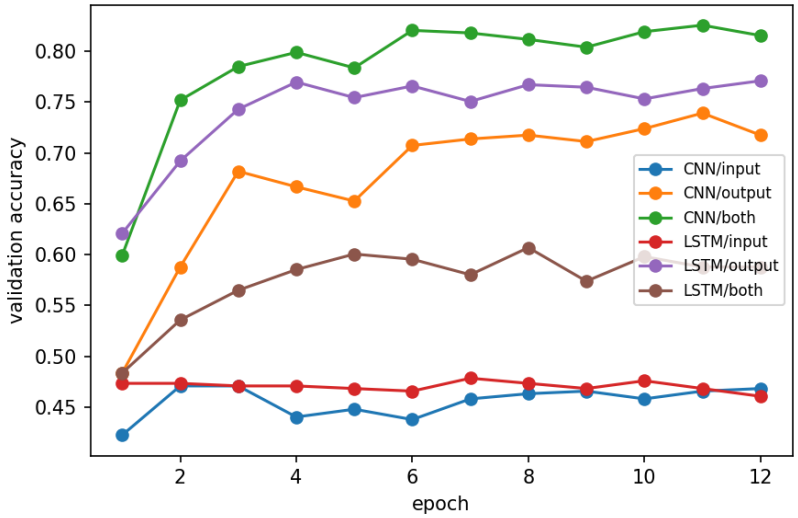
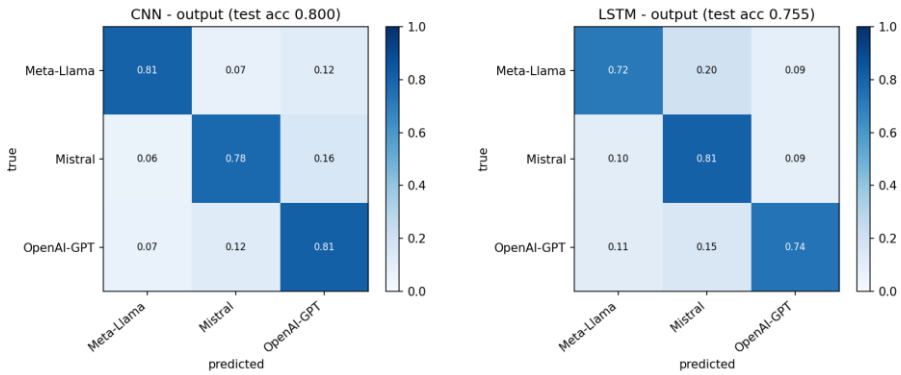


Figure 1: Validation Accuracy per Epoch

A few of the runs benefited much more from longer periods of training. LSTM/output, LSTM/both, and CNN/both effectively capped within the first 4-6 epochs, whereas CNN/output appeared to continue improving up until epoch 11. Finally, CNN/input and LSTM/input stagnated near 45% from the very beginning, greater than random guessing, (33%) but clearly there was no deeper information to gain from the inputs.

Section 4: RQ1 – Can we identify which LLM generated a response?



Figures 2 and 3: Output-Only Confusion Matrices (CNN left, LSTM right)

model	family	CNN	LSTM
llama-chat	Meta-Llama	0.808	0.717
mistral	Mistral	0.888	0.931
mistral-chat	Mistral	0.672	0.690
chatgpt	OpenAI-GPT	0.712	0.479
gpt3	OpenAI-GPT	0.767	1.000
gpt4	OpenAI-GPT	0.959	0.740
overall		0.800	0.755

Table 3: Output-Only Test Accuracy for Each Individual Model

Analysis: From the table and confusion matrices, both CNN and LSTM perform well. They are both able to identify the family that generated a given response with at least 72% accuracy for all families. Overall, CNN performs slightly better, with an overall average of 80% as opposed to the 75.5% accuracy of LSTM.

One notable oddity is the discrepancy between LSTM's ability to identify gpt3 and chatgpt. Chatgpt is identified as an OpenAI model with an accuracy of 47.9%, whereas gpt3 is identified 100% of the time. From looking at the data, gpt3's outputs always begin with a line break, something that none of the other outputs do. The reason that LSTM can pick up on this fact while CNN cannot might be because it reads in both directions, with its backwards pass ending exactly on the newline, whereas CNN takes the max of all locations, likely missing this.

Chatgpt is likely less accurately identified by LSTM because of the three chatbot models, it is the least prevalent, having only 133 outputs per domain, whereas mistral-chat has 200, and llama-chat has 400. Because of this, features that could be attributed to chatgpt might instead be attributed to the other two LLM families, or even gpt4, which is also a chat model.

Section 5: RQ2 – Does the user's prompt help identify the LLM?

model	features	acc	macro_f1
CNN	input	0.512	0.401
CNN	output	0.800	0.800
CNN	both	0.861	0.861
LSTM	input	0.512	0.457
LSTM	output	0.755	0.756
LSTM	both	0.669	0.665

Table 4: Full Input/Output Comparison

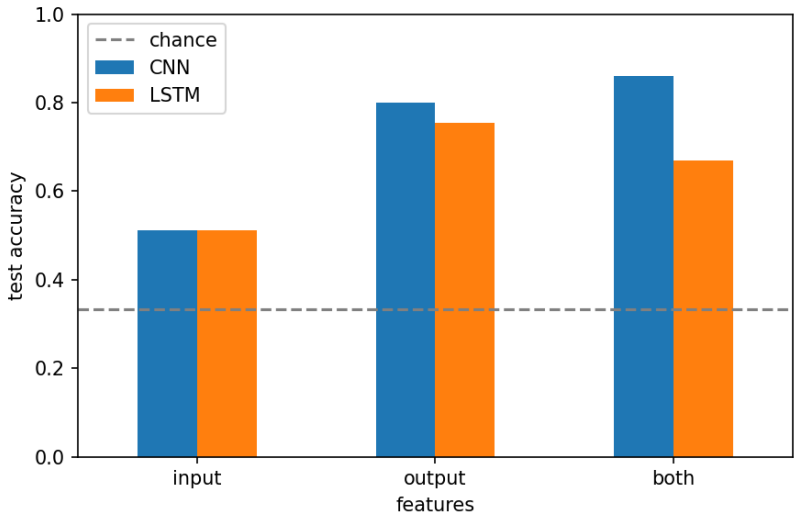
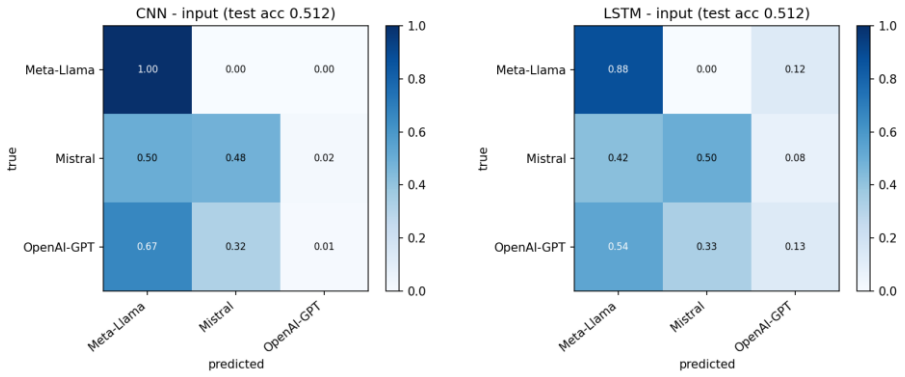


Figure 4: Test Accuracy by Feature Set (dashed line = chance)



Figures 5 and 6: Input-Only Confusion Matrices (CNN left, LSTM right)

Analysis: CNN and LSTM both almost perfectly identify llama. This is likely because the only llama model is the chat model. All inputs to chat models in the RAID dataset begin with "Write the…", whereas all inputs to non-chat models begin with "The following is the full text of the…". Because of this, both CNN and LSTM most likely always pick Meta-Llama as the family when it sees this input, which is 100% right in llama-chat's case, but causes Mistral and GPT to be chosen with far less accuracy.

When including both the input and output, CNN tends to benefit, likely due to learning the difference between completion and chat models, while LSTM tends to decrease in performance. I would guess that the reason LSTM performs worse with the input is the same reason it

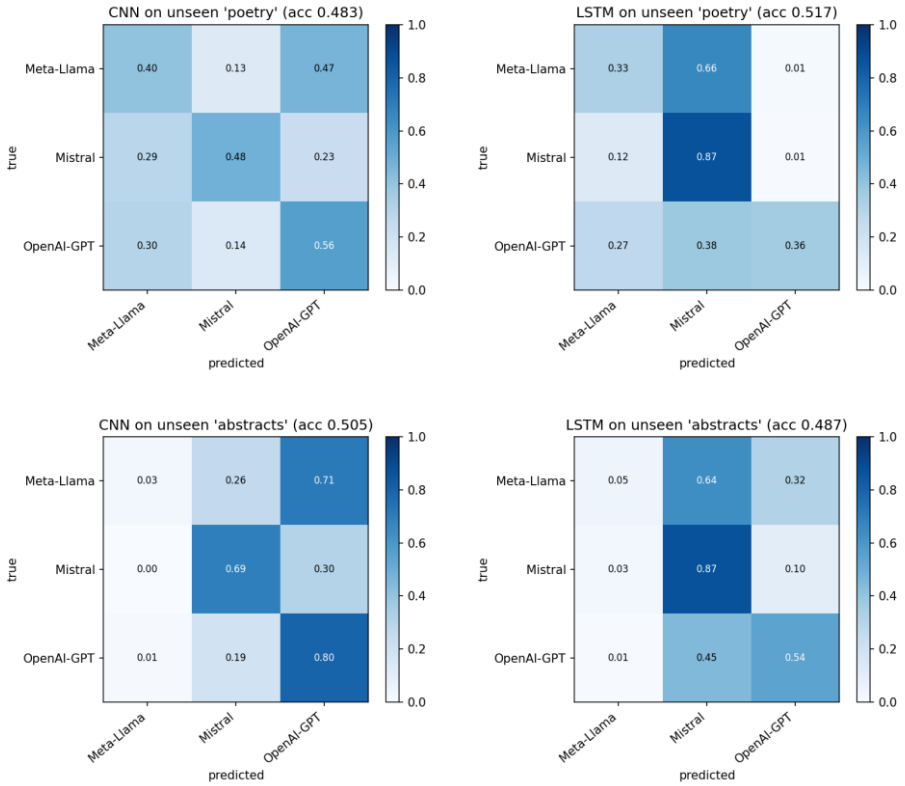
performed so well without it. The line break that identified gpt3 is now in the middle of the combined sequence, which gets rid of the advantage of the two-directional reading.

In general, due to the way that the RAID dataset has been created, including the input adds more confounding variables and RQ1 is likely the more accurate test of efficacy.

Section 6: RQ3 – Do LLM fingerprints generalize across tasks or domains?

held-out domain	model	in-domain acc	unseen-domain acc	drop	unseen macro F1
poetry	CNN	0.718	0.483	0.235	0.485
poetry	LSTM	0.656	0.517	0.139	0.498
abstracts	CNN	0.892	0.505	0.386	0.421
abstracts	LSTM	0.806	0.487	0.319	0.410

Table 5: In-Domain vs. Unseen-Domain Accuracy



Figures 7-10: Confusion Matrices on Unseen Poems (top) and Abstracts (bottom) (CNN left, LSTM right)

Analysis: In this test, each model is trained in 3 out of 4 domains and then tested on the last domain to see if they have learned to recognize outputs for a given family or outputs from a

given domain. In general, these models can identify the correct family from these outputs at a lower rate than the models from RQ1, falling to about 50% across the board. This implies that much of the training went towards identifying the way each model generates output in each domain, not how it generates text broadly.

From a glance, LSTM appears to be able to identify Mistral-written text in previously unseen domains. However, upon closer inspection, it appears that it is using Mistral as a default guess, choosing it most frequently for most families. In fact, in 28% of mistral (non-chat) outputs, the output is preceded with ">". This irregularity, which LSTM may pick up on better due to its backwards pass, might have signaled to the model that Mistral is the family that generates unfamiliar, heterogenous outputs, and therefore it becomes the default guess. Furthermore, a likely reason that LSTM does not pick Mistral to a greater degree on OpenAI-GPT models is the line breaks that preceded every gpt3 output and would still hold true for previously unseen domains

Furthermore, CNN appears to largely attribute Llama-generated text in new domains to OpenAI, both when exposed to previously unseen poetry and abstracts. Since CNN is trained to find 3–5-word phrases using similar vocabulary, it likely learns to identify certain word choices in the domain of one model that it incorrectly attributes to the same model under a different domain. One example of this is that 72% of OpenAI outputs from the Book domain use the word "novel," whereas 91% of llama outputs from the Abstract domain use the word "novel" in an entirely different context. These superficial similarities in vocabulary likely lead to many Llama outputs being attributed to GPT.

Overall, I would conclude that LLM fingerprints do not generalize over domains very well, at least within the context of my data. It appears that the CNN tends to learn the vocabulary, and the LSTM tends to learn the form, of previous domains, neither of which apply cleanly.

Section 7: RQ4 – What characteristics distinguish different LLMs?

family	tokens	type-token ratio	words per sentence	avg word length	punctuation per 100 tokens	line breaks
Meta-Llama	332.87	0.57	38.76	4.89	12.49	14.46
Mistral	277.44	0.58	38.14	4.78	11.93	14.69
OpenAI-GPT	253.95	0.59	23.98	4.88	12.95	12.96

Table 6: Average Output Statistics per Family

Meta-Llama	Mistral	OpenAI-GPT
their	abstract	the
despite	very	this paper
that	how	the novel
has	at	segmentation
bright	some	paper presents
or	the paper	experimental results
including	another	silent
several	utilizes	paper

Table 7: Most Distinctive Words and Phrases per Family (TF-IDF + Logistic Regression)

ablation	CNN	LSTM
original	0.800	0.755
first 32 tokens	0.524	0.712
lowercased	0.766	0.712
no punctuation	0.650	0.660
no line breaks	0.693	0.669
word order shuffled	0.511	0.436

Table 8: Output-Only Test Accuracy under Each Ablation

Analysis: Overall, several differences between models exist. Table 6 shows that Meta tends to generate more tokens per output than either Mistral or OpenAI, and Meta and Mistral both use a similar raised number of words per sentence compared to OpenAI.

Table 7 does not provide much insight into the differences in vocabulary between the three families, with most words being rather average. However, it appears that Mistral and OpenAI tend to repeat the domain of their prompt at a much higher rate than Meta does, which could be one explanation for the lower accuracy of identifying new domains within RQ3.

Table 8 supports a few previous hypotheses- LSTM still performs quite well when only given the first 32 tokens of an output, whereas CNN performs quite poorly. Given that much of the giveaways within the outputs, such as line breaks or ">", are in the first 32 tokens, it makes sense that LSTM does not see much drop. No other ablations show much of a disparity between the two models, but it does appear that certain signals such as capitalization, punctuation, line breaks, and word order do play some role in determining the accuracy of these models in recognizing each LLM family, as removing these signals decreases both models roughly equally.

Section 8: Lessons & Experience

A major lesson I learned from this project is with how much the sampling of data within a data set matters. Initially, I only sampled the first 400 outputs from every family for every domain, and my scores for RQ1 were much higher, scoring in the 90s across the board. However, when I tried RQ2, it became clear that something was wrong with my data. Inputs to Meta models were

being identified with 100% accuracy, whereas inputs to OpenAI and Mistral models were being attributed to OpenAI 100% of the time, leading to an accuracy of 66% no matter how long the models trained on it. Upon looking into the data I realized that my 400 samples were only taking the first models from each family- llama-chat, mistral, and gpt3. Since only one model was a chat-based model, there was a major giveaway that the models were taking advantage of. As a result, I decided to sample models within each family evenly, which led to my current results.

However, as mentioned within RQ1, the quantity of data from each model has also differed as a result, leading to certain models being less recognizable in comparison to others. When training models in the future, I will take more care in my sampling of data so confounding variables such as this do not appear.

Another lesson I learned was the value of training models until you are certain they are done learning. Initially, I set the training period to 5 epochs, which showed LSTM outperforming CNN overall. All models looked to be mostly settled, with CNN even showing a slight dropoff in accuracy. However, on the off chance that one of the models might be undertrained, I increased the number of epochs gradually, which showed CNN continue to improve until 12 epochs.

Section 9: Works Cited:

Dugan, Liam. *RAID*. Hugging Face, 2024, huggingface.co/datasets/liamdugan/raid. Accessed 11 Oct. 2026.